role, as long as the relationship graph is updated to reflect their new role. There's no need to modify policies or update ACLs because access is determined by the data itself.

8.3.3 Applying the PARC Model to OpenFGA

OpenFGA takes a fundamentally different approach from languages like XACML, Rego, or Cedar. Instead of operating over attributes or evaluating logic directly, it defines authorization through relationships between typed objects—like users, resources, groups, and roles—represented as data in a graph. Permissions are not written as rules in code but are declared as logical expressions over these relationships and enforced via graph traversal. This approach makes OpenFGA especially suitable for applications like ACME's Customer Collaboration platform, where sharing, delegation, and organizational relationships play a key role in access decisions.

Here's how OpenFGA maps to the PARC model:

- **Principal**—Principals in OpenFGA are usually instances of a user type, referenced in tuples such as `document:123#viewer@user:alice`. There are no attributes or profiles; principals are identified only by their presence in relationships. This simplifies the model but requires identity-related logic (like group membership or department) to be explicitly modeled in the graph.
- **Action**—OpenFGA doesn't have actions in the traditional sense. Instead, permissions are seen as named access checks, such as `can_view`, `can_edit`, and `can_access`. These permissions aren't just strings in a request payload; they are predefined in the schema as logical combinations of relationships. This approach fits well with applications that have fixed sets of actions and clear semantics.
- **Resource**—Resources are first-class, typed entities such as a document, folder, or report. Each resource instance has a set of relationship tuples stored in the system, defining who can do what. The schema strictly controls which relationships and permissions are valid for each resource type, providing strong guarantees about how authorization logic is implemented.